

Arquitectura Profunda y Especificaciones

Técnicas: Motor Multi-Agente

DeepResearcher

1. Introducción y Filosofía de Diseño

El módulo `DeepResearcher` representa el estado del arte en la orquestación de agentes autónomos dentro del ecosistema KognitoAI. A diferencia de los sistemas RAG (Retrieval-Augmented Generation) tradicionales que realizan búsquedas semánticas simples (one-shot), `DeepResearcher` implementa un flujo de trabajo iterativo, jerárquico y paralelo. Utiliza la biblioteca **LangGraph** para modelar el proceso de investigación como un grafo de estados cíclico, permitiendo que el sistema planifique, delegue, ejecute, evalúe y sintetice información de manera autónoma.

Este documento técnico desglosa cada componente del sistema, desde la gestión de memoria y el tipado estricto, hasta los mecanismos de resiliencia ante fallos de red y las estrategias de optimización de la ventana de contexto (Token Pruning).

2. Modelado de Estado y Tipado Estricto (`deep_researcher_state.py`)

La base de la estabilidad del sistema radica en su gestión del estado. `LangGraph` pasa un objeto de estado entre los nodos del grafo. En `DeepResearcher`, este estado está fuertemente tipado utilizando `TypedDict` de Python y modelos `BaseModel` de Pydantic.

2.1. El Reductor Personalizado (`override_reducer`)

Por defecto, `LangGraph` utiliza `operator.add` para actualizar el estado, lo que significa que si un nodo devuelve una lista de notas, estas se concatenan a las existentes. En un sistema iterativo, esto provoca una duplicación masiva de datos y el agotamiento rápido de la ventana de contexto. Para solucionar esto, se implementó un reductor inteligente:

```
def override_reducer(current_value, new_value):
    if isinstance(new_value, dict) and new_value.get("type") == "override":
        return new_value.get("value", new_value)
    else:
        if current_value is None:
            return new_value if new_value is not None else []
        if new_value is None:
            return current_value
        return operator.add(current_value, new_value)
```

Explicación Técnica: Este reductor permite que un nodo decida dinámicamente si desea hacer un `append` (comportamiento por defecto) o un `override` completo del estado. Por ejemplo, el nodo `compress_research` utiliza `{"type": "override", "value": deduplicated_sources}` para reemplazar la lista de fuentes con una versión limpia y sin duplicados, manteniendo la memoria del grafo optimizada.

2.2. Aislamiento de Estados por Subgrafo

Para permitir la ejecución paralela sin condiciones de carrera (race conditions) en el historial de mensajes, el sistema define estados aislados:

- **AgentState** : El estado global (Root). Mantiene el `research_brief` final y el `final_report`.
- **SupervisorState** : Estado local para el orquestador. Mantiene sus propios `supervisor_messages`.
- **ResearcherState y ExpertAgentState** : Estados efímeros para los trabajadores. Cada trabajador tiene su propia lista de `researcher_messages`, lo que significa que un Experto Financiero no se confunde con el historial de búsqueda de un Experto Legal que se está ejecutando en paralelo.

2.3. Structured Outputs (Tool Calling)

Se utilizan clases Pydantic para forzar a los LLMs a emitir decisiones estructuradas. Por ejemplo, la clase `CreateExpertAgent` obliga al Supervisor a definir:

- `expert_name` : El rol (ej. "Analista de Ciberseguridad").
- `expert_specialty` : El dominio de conocimiento.
- `custom_prompt_instructions` : Instrucciones inyectadas dinámicamente en el System Prompt del trabajador.
- `research_depth` : Nivel de profundidad requerido.

3. Arquitectura del Grafo y Patrones Multi-Agente (deep_researcher.py)

La ejecución se modela mediante grafos anidados.

3.1. El Grafo Principal (Root Graph)

1. `clarify_with_user` : Actúa como un guardián (gatekeeper). Utiliza el modelo `ClarifyWithUser` para evaluar si el *prompt* inicial del usuario es ambiguo. Si `need_clarification` es `True`, el grafo se interrumpe y pide más datos.
2. `write_research_brief` : Convierte la conversación coloquial en un objetivo de investigación determinista y estructurado.
3. `research_supervisor` : Invoca el subgrafo del Supervisor.
4. `final_report_generation` : Nodo terminal que consolida todos los datos comprimidos en un documento Markdown.

3.2. Paralelismo Masivo en el Subgrafo del Supervisor

El Supervisor analiza el *brief* y decide qué herramientas usar. El verdadero poder técnico reside en el nodo `supervisor_tools`. En lugar de ejecutar las herramientas secuencialmente, el código agrupa las llamadas a `ConductResearch` y `CreateExpertAgent` y las lanza concurrentemente:

```
# Ejecución concurrente de subgrafos de investigación
results = await asyncio.gather(*conduct_research_tasks, *expert_agent_tasks,
return_exceptions=True)
```

Explicación Técnica: El uso de `return_exceptions=True` es crítico para la tolerancia a fallos. Si el sistema lanza 5 agentes expertos y uno de ellos sufre un error de Timeout (HTTP 408) o de Rate Limit (HTTP 429), la excepción es capturada y devuelta como un objeto en la lista `results`, permitiendo

que los otros 4 agentes completen su trabajo y devuelvan datos valiosos. El sistema simplemente registra el error del agente fallido y continúa.

4. Ecosistema de Herramientas y Optimización de Contexto (deep_researcher_utils.py)

El manejo eficiente de la información web es el mayor desafío en agentes de investigación.

4.1. Compresión de Búsqueda al Vuelo (tavily_search)

Cuando un agente busca en la web, obtener el contenido crudo de 5 páginas web podría consumir fácilmente 50,000 tokens, saturando el LLM investigador. Para evitar esto, `tavily_search` implementa un patrón Map-Reduce asíncrono:

1. Realiza la búsqueda vía API (`tavily_search_async`).
2. Itera sobre las URLs devueltas y lanza tareas simultáneas a un **LLM rápido y económico** (ej. Gemini Flash o Claude Haiku) utilizando la función `summarize_webpage` .
3. Este LLM rápido extrae un resumen conciso y hasta 5 citas clave (utilizando el esquema Pydantic `Summary`).
4. El agente investigador solo recibe estos resúmenes altamente densos, reduciendo el consumo de tokens en un 90% y mejorando la precisión (Signal-to-Noise ratio).

4.2. La Herramienta de Reflexión (deep_research_think_tool)

Es una herramienta "No-Op" (No Operation) a nivel de sistema, pero vital a nivel cognitivo. Permite al LLM implementar el patrón *Chain of Thought* (CoT). Antes de comprometerse a una búsqueda costosa, el LLM llama a `think_tool(reflection="...")` para escribir sus suposiciones, evaluar qué datos le faltan y planificar sus siguientes pasos.

5. Pipeline de Deduplicación y Trazabilidad de Citas

Para que el informe final sea académicamente riguroso, las fuentes deben ser precisas y no estar duplicadas.

5.1. Identificadores Estables (MD5)

Cada vez que una herramienta devuelve una fuente, la URL pasa por `generate_stable_id()` :

```
hash_obj = hashlib.md5(url.encode('utf-8'))
short_hash = hash_obj.hexdigest()[:12]
return f"src_{short_hash}"
```

Esto garantiza que si el "Investigador A" y el "Experto Financiero B" encuentran el mismo artículo de Bloomberg de forma independiente, ambos generarán exactamente el mismo ID (`src_8f7b3a1c...`).

5.2. Fusión de Metadatos en el Nodo de Compresión

El nodo `compress_research` recopila todos los `ToolMessages` . Utiliza expresiones regulares para extraer URLs de textos libres y parsea JSONs estructurados. Al detectar IDs estables duplicados, fusiona la información, combinando los `tool_names` para que el sistema sepa que esa fuente fue validada por múltiples vectores de búsqueda.

6. Resiliencia, Fallbacks y Prevención de Colapsos

Los LLMs son sistemas no deterministas propensos a fallos bajo carga. KognitoAI implementa múltiples capas de protección:

6.1. Fallback de Tool Choice (OpenRouter Bug Mitigation)

Algunos proveedores fallan al procesar el parámetro `tool_choice` estricto en modelos de código abierto. El sistema envuelve las llamadas estructuradas en un bloque `try-except`. Si detecta un error de `tool_choice`, hace un fallback dinámico a `method="json_mode"`, inyectando el esquema en el prompt del sistema y parseando la respuesta manualmente.

6.2. Protección de Ventana de Contexto (Token Pruning)

La función `prune_messages_to_fit_token_limit` se ejecuta antes de cada llamada al LLM. Calcula los tokens actuales y, si superan el umbral seguro (ej. 100,000 tokens definidos en `Configuration`), elimina iterativamente los mensajes más antiguos del medio de la conversación, preservando siempre el *System Prompt* (instrucciones base) y los últimos mensajes (contexto inmediato).

6.3. Recuperación de Caídas a Medio Flujo (`MidStreamFallbackError`)

En la generación del reporte final, que puede requerir miles de tokens de salida, las conexiones suelen sufrir timeouts. Si ocurre un fallo, el sistema entra en un bucle de reintento con *Backoff Exponencial* (`_retry_delay *= 2`). En los reintentos posteriores, recorta agresivamente la ventana de contexto de entrada para aliviar la carga de cómputo del proveedor de LLM, aumentando las probabilidades de éxito.

7. Ingeniería de Prompts de Grado Producción (`deep_researcher_prompts.py`)

Los prompts no son simples instrucciones; son arquitecturas de comportamiento.

7.1. Inyección de Personalidad Dinámica

El `expert_agent_system_prompt` utiliza interpolación de strings para crear un rol hiper-específico en tiempo de ejecución. Al inyectar `{expert_specialty}` y `{custom_prompt_instructions}`, el modelo altera su espacio latente para priorizar terminología, heurísticas y fuentes específicas de ese dominio.

7.2. El Mandato Anti-Resumen

El `compress_research_system_prompt` tiene una directiva psicológica contraintuitiva: **"Su trabajo NO es resumir, sino ESTRUCTURAR Y PRESERVAR"**. Se le prohíbe explícitamente acortar la información. Su única tarea es limpiar el ruido y agrupar los datos duros bajo la fuente correspondiente. Esto garantiza que el redactor final tenga materia prima densa y no una "fotocopia de una fotocopia".

7.3. Renderizado Avanzado (Markdown + HTML/CSS)

El `final_report_generation_prompt` exige un documento de nivel "White Paper" ejecutivo.

- Exige el uso de **Markdown puro** para la estructura narrativa.
- Exige citas estrictas en formato numérico `[N]`.

- **Innovación UI:** Obliga al LLM a generar un bloque XML `<visual_schema>...`
`</visual_schema>` al final del documento. Dentro de este bloque, el LLM debe escribir **HTML5 y CSS3 inline** (Flexbox, gradientes, tarjetas, sombras) para crear una representación visual de los hallazgos. El frontend extrae este bloque y lo renderiza de forma nativa.

8. Configuración y Ajuste Fino (`deep_researcher_config.py`)

La clase `Configuration` expone los hiperparámetros del sistema:

- `max_concurrent_research_units` : Define el grado de paralelismo (default: 8).
- `max_researcher_iterations` : Límite de profundidad recursiva para evitar bucles de investigación infinitos.
- **Modelos Desacoplados:** Permite configurar un modelo masivo/lento para `final_report_model` (ej. GPT-4o o Claude 3.5 Sonnet) y modelos rápidos/baratos para `summarization_model` (ej. Gemini 2.0 Flash), optimizando la relación costo-rendimiento de la investigación.